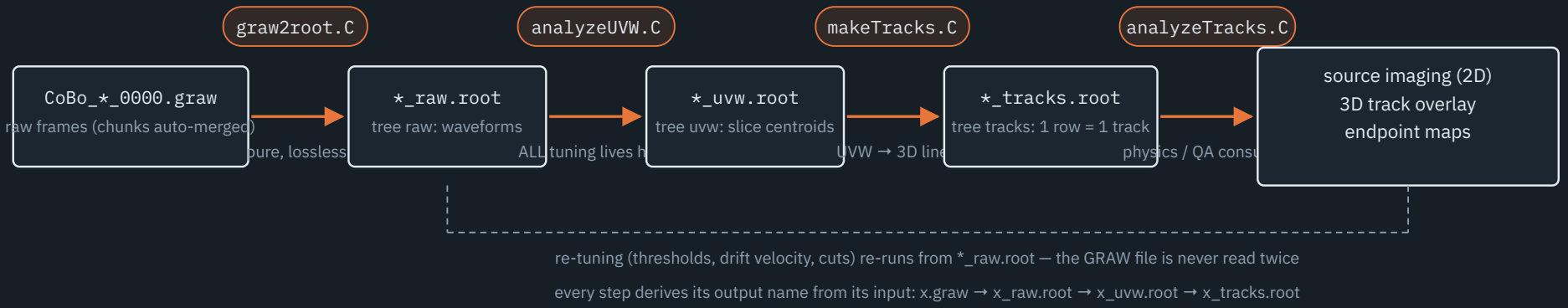
MINI ETPC / STANDALONE MACROS

mini eTPC Analysis Macros

From raw GRAW files to 3D tracks and source imaging — a four-step macro chain that needs only ROOT 6 (no GET software, no TPCReco)



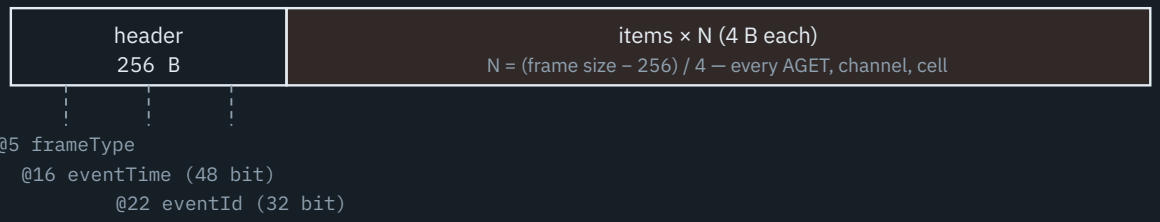
Three working assumptions: ① graw2root.C does zero analysis (no pedestal, no threshold, no geometry) — conversion is lossless ② one event = one straight alpha track (mini eTPC running condition) ③ the z scale comes from a drift velocity computed from the run conditions (details on slide 3)

STEP 1 — GRAW2ROOT.C

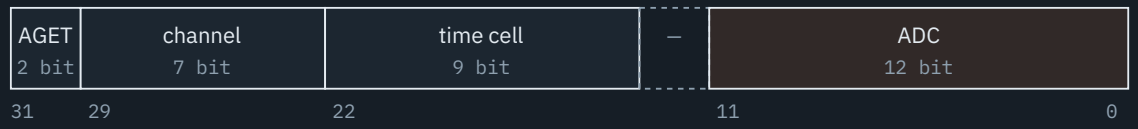
GRAW frames straight into a ROOT tree

CoBo full-readout frames (frameType 1, revision 5, single AsAd) are decoded directly — no GET software involved

one frame = one event



item bit layout (32 bit, big-endian)



→ unpacked verbatim into adc[AGET][channel][cell] (4 × 68 × 512)

frameType ≠ 1 (e.g. 12 B topology frames) is skipped; chunks _0001, _0002 … are merged automatically

eventId / eventTime come from the header, waveforms from the items. Nothing is subtracted or selected.

Output: tree "raw" (1 entry = 1 event)

`eventId`	event number from the frame header
`eventTime`	48-bit CoBo timestamp (clock ticks)
`adc[4][68][512]`	raw ADC of every waveform — **FPN included, nothing subtracted**

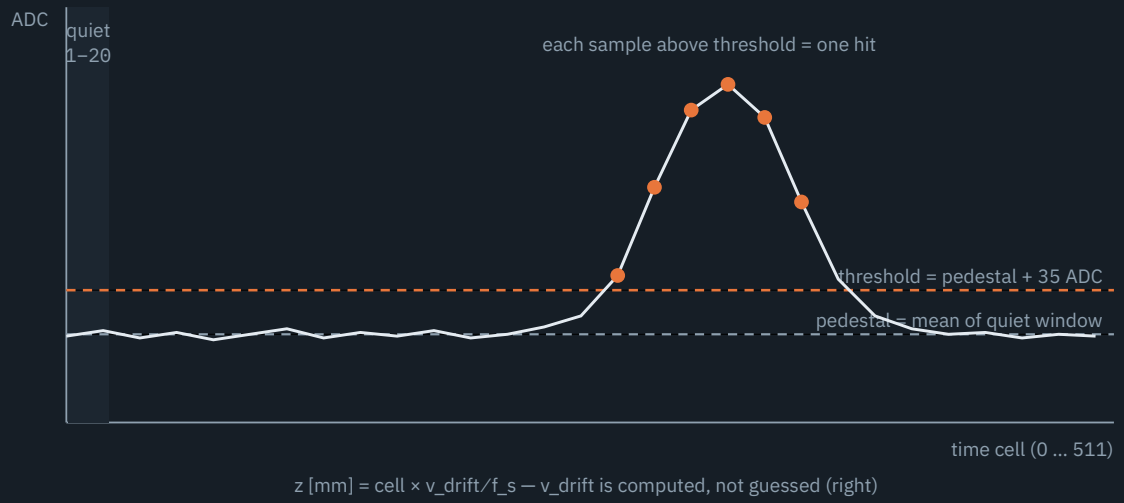
Input: the first chunk (followers are auto-discovered) or a text file listing GRAW paths. Output: one file per run.

```
root -l -b -q 'graw2root.C+("CoBo_xxx_0000.graw")'
```

STEP 2A — ANALYZEUVW.C (STAGE 0)

Waveforms → hits: pedestal, threshold, channel map, z scale

The first appearance of physical coordinates — and the home of every tuning constant



z [mm] = cell × v_drift/f_s — v_drift is computed, not guessed (right)

Per channel: pedestal from a quiet window, then every sample above pedestal + 35 ADC becomes one hit (z, s, q).

Channel → physical coordinate

`channel_map.csv` maps (AGET, ch) → (view, strip position s in mm): **U 72** / **V 92** / **W 92** strips, 1.5 mm pitch. FPN channels {11, 22, 45, 56} are absent from the map and drop out automatically.

Drift velocity from run conditions

```
v_drift = 0.7233 × (E ⁄ p) [cm/μs]
E = HV ⁄ L,  p in mbar
```

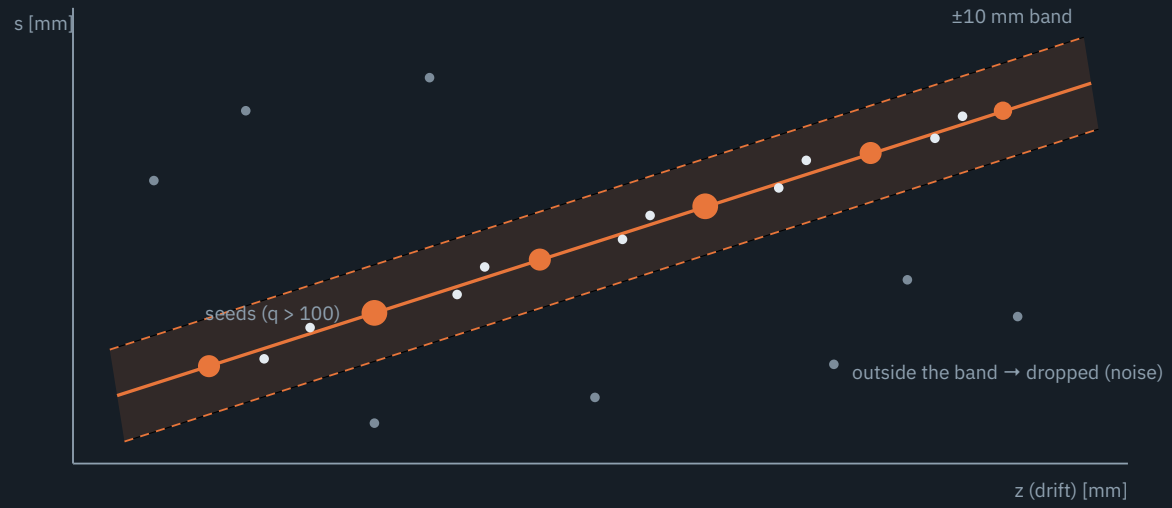
The coefficient is the CO_2 mobility constant fitted to a Magboltz-derived operating-point table (7 points, 130–250 mbar, spread ±0.2%). Valid for E/p = 0.54–1.05 V/(cm·mbar); the macro warns outside this range. 1600 V / 20 cm / 100 mbar → **0.579 cm/μs**.

> Table-derived, not measured — confirm with the z-edge method before trusting absolute lengths.

STEP 2B — ANALYZEUVW.C (STAGES 1–3)

Seed a line per view, keep a band, measure the width

The single-track assumption enters here — once per view, independently for U, V, W



Big dots: seed hits (charge-weighted 2D PCA fixes the line). Small black: kept by the band. Grey: rejected.

Per view (U, V, W independently)

- **1. Seed**: charge-weighted PCA on hits with q > 100 only → line direction
- **2. Band**: keep hits within ±10 mm perpendicular distance of the line
- **3. Width**: charge-weighted RMS of the perpendicular residuals → `width` (diffusion proxy)

Written to the uvw tree

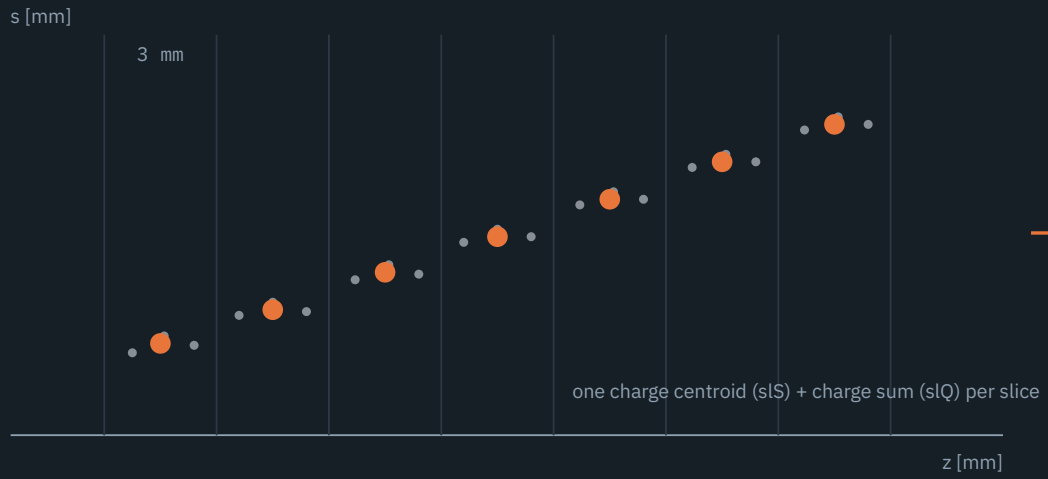
`qview[3]`	total accepted charge per view
`width[3]`	track width per view [mm]
`nhits[3]`	accepted hit count

Views with fewer than 10 hits, or no seed charge, are skipped (values stay 0).

STEP 2C — ANALYZEUVW.C (STAGE 4)

Slice the drift axis on a global 3 mm grid

The interface between analyzeUVW and makeTracks: one charge centroid per (view, slice)



four parallel vectors (same length & order)

slView	view: 0=U 1=V 2=W
slZ	slice CENTER z = (index + 0.5) × 3 [mm]
slS	charge-weighted strip-coordinate centroid [mm]
slQ	summed band-selected charge [ADC]

grid = floor(z / 3 mm) — independent of event and view, so the U/V/W centroids of the same physical z interval share an index. That alignment is what makes 3D possible.

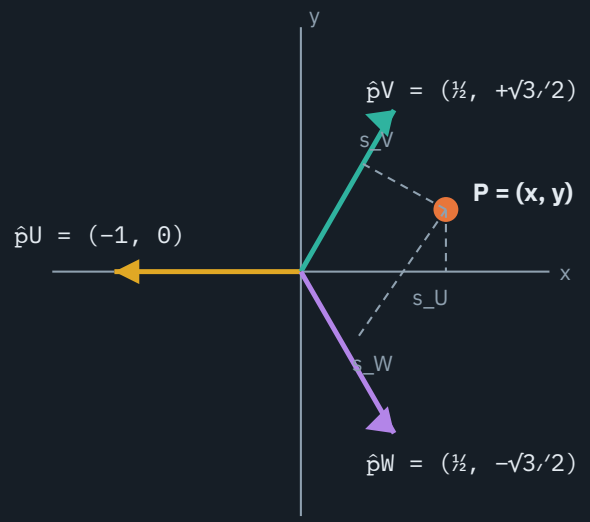
Only band-accepted hits contribute. Every (view, slice) with charge appends one entry to the four vectors.

STEP 3 — MAKETRACKS.C (PART 1 OF 2)

3D reconstruction I — stereo triangulation per slice

Three 1D strip coordinates measured at the same z become one (x, y) point

within one slice: solve $s = \hat{p} \cdot (x, y)$



Each view measures one projection of P. Two views already fix P; the third over-determines it (a built-in consistency check).

Exactly what the code does

- Regroup the slice vectors by `floor(slZ / 3)`
- For every slice seen by **≥ 2 views**, accumulate the 2×2 normal equations and solve

$$A = \Sigma_v \hat{p}_v \hat{p}_v^{\top}, \quad b = \Sigma_v \hat{p}_v s_v$$
$$(x, y) = A^{-1} b \quad \text{(skip if } \det A \approx 0\text{)}$$

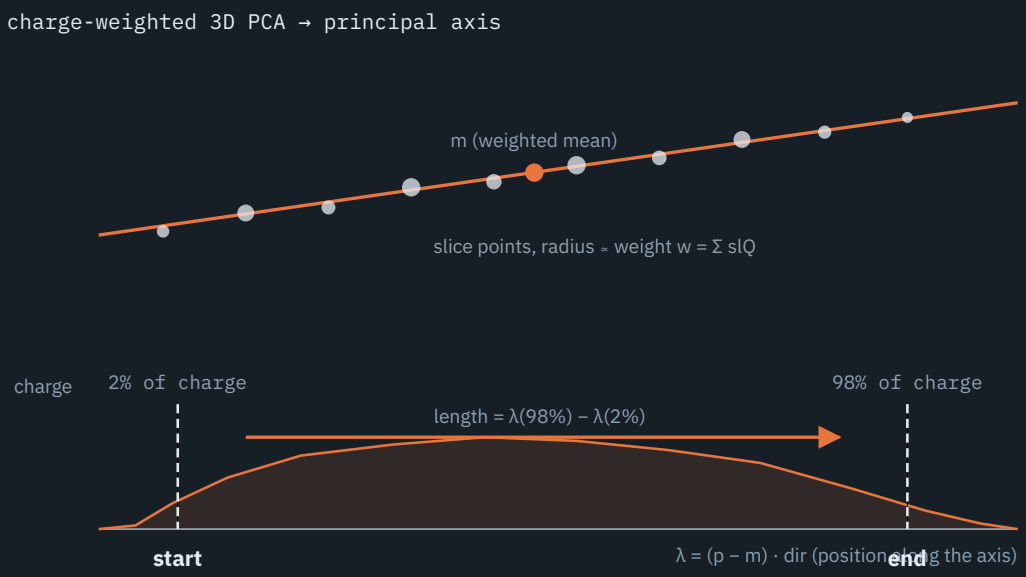
- **z** of the point = charge-weighted mean of the slice's `slZ`
- **weight** of the point = Σ `slQ` (used by the line fit next)
- Slices seen by only one view cannot fix (x, y) and are dropped — a track clipping a detector corner loses its tip

Result: an ordered chain of charge-weighted 3D points (x, y, z, w) — typically one per 3 mm of drift.

STEP 3 — MAKETRACKS.C (PART 2 OF 2)

3D reconstruction II — line fit and charge-quantile endpoints

A charge-weighted principal axis through the slice points, endpoints from the charge profile along it



Top: the fitted axis. Bottom: the charge profile along it; robust endpoints are the 2% / 98% charge quantiles, not the extreme points.

The fit, step by step

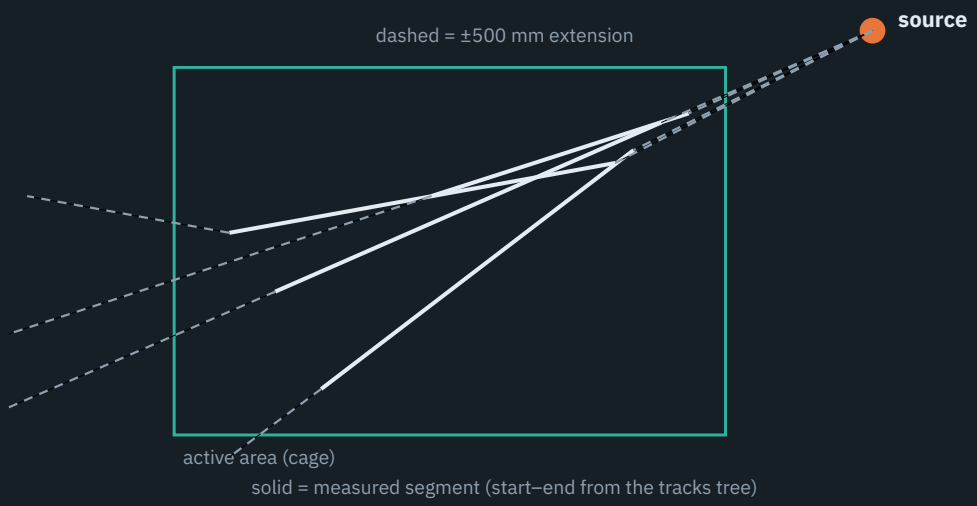
- Weighted mean m and covariance $C = \Sigma\, w\,(p-m)(p-m)^{T}$ over the slice points
- **dir** = eigenvector of C with the largest eigenvalue (`TMatrixDSymEigen`)
- Project each point: $\lambda = (p - m)\cdot dir$; sort; walk the cumulative charge to the 2% and 98% quantiles → start / end (robust against stray slices)
- `length` = $\lambda_{98} - \lambda_{2}$, `charge` = Σ w, `dEdx` = charge/length (the gain metric)
- < 3 slice points → `ok = 0` (no fit)

> **Head/tail is a coin flip.** The code labels the charge-heavier half "end", but mini eTPC alphas cross without stopping — no Bragg peak, so this orientation carries no
>
> information (measured ≈ 50% on contained samples). Use |dir| or fix the sign from the source geometry.

STEP 4 — ANALYZETRACKS.C (PART 1 OF 2)

Source imaging by unsigned back-projection

Extend every track beyond both ends; a point source lights up where the lines converge — even outside the cage



Lines from a common origin converge when extended — head/tail ignorance does not matter, which is why the method works here.

Accumulation recipe (the three tricks)

- **Quality cut first**: `ok && length > 50 mm` — failed fits are stored as all-zero rows and must not enter
- **Arc-length stepping**: walk the extended segment in ~1 mm steps (never parametrise by x — near-vertical tracks vanish and the density becomes angle-biased)
- **One track = total weight 1**: deposit 1/n_steps per step, so a long line and a short one get the same vote — this is what makes the convergence blob stand out

Histogram hygiene

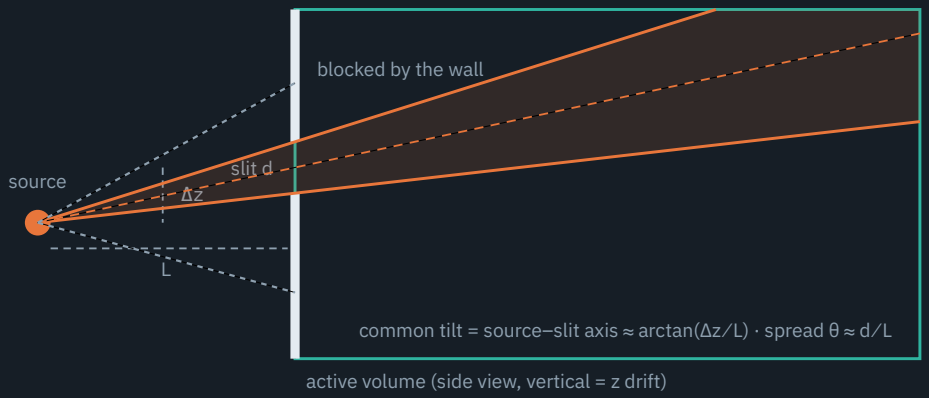
2 mm bins with 1 mm steps (≥ 2 samples per bin) — a bin width at or below the step size produces aliasing stripes. The cage outline is overlaid for orientation.

STEP 4 — ANALYZETRACKS.C (PART 2 OF 2)

3D overlay — and why the source tracks all share one tilt

Every selected track as a 3D segment (rotatable); the uniform tilt is collimation geometry, not a field effect

side view: source + slit at a DIFFERENT height = tilted collimator



The slit sits at a different height (z) than the source, so the accepted cone points along the tilted source–slit axis: in the XZ projection nearly every source track shows the same angle, with a spread of only θ ≈ d/L around it.

Why it is NOT the drift field

- A 5.5 MeV alpha in an 80 V/cm field: the potential across the chamber is ~10 eV — six orders of magnitude below the kinetic energy. No bending.
- A *uniform* drift field maps straight tracks to straight tracks with the same slope: z images the true ionisation height.
- Field distortions do exist near cage openings — but they produce a local *kink* near the entrance, not a uniform tilt of the whole track. Check the fit residuals near the entry side to separate the two.

Built-in test

The dashed 3D extensions should converge on

one point = the source. Caveat for a vertex fit: a collimated (nearly parallel) bundle fixes the vertex well *across* the beam axis but poorly *along* it — expect an error ellipse elongated toward the source.

REFERENCE

Tuning constants and known pitfalls

All constants sit at the top of their macro — change one, re-run from *_raw.root, never re-read the GRAW

constant	default	meaning
DRIFT_HV_V	1600	drift cage voltage [V] (v_drift is computed)
DRIFT_LEN_CM	20	drift length [cm]
PRESSURE_MBAR	100	CO_2 pressure [mbar]
SAMPLING_MHZ	25	AGET sampling rate
ADC_THRESHOLD	35	hit threshold above pedestal [ADC]
PED_LO / PED_HI	1 / 20	pedestal quiet window [cells]
SEED_ADC	100	minimum charge for line-seed hits
BAND_MM	10	acceptance band half-width [mm]
SLICE_MM	3	drift-axis slice width [mm]
MIN_SLICES / QUANTILE	3 / 0.02	(makeTracks) min 3D points; endpoint quantile
MIN_LENGTH_MM / EXTEND_MM	50 / 500	(analyzeTracks) quality cut; back-projection extension

Pitfalls

- **z is only as good as v_drift** — the mobility law v = 0.7233 (E/p) is interpolated from an operating-point table, not measured. Cross-check with the z-edge method; a wrong v_drift scales all z lengths and tilt angles.
- **The 3 mm grid lives in two places** — `SLICE_MM` in analyzeUVW.C and a literal 3.0 in makeTracks.C. Change one, change both.
- **Head/tail carries no information** here (no Bragg peak). Orient tracks from the source geometry.
- **A single-view slice is dropped** — corner-clipping tracks lose their tip.
- **frameType 2 (compact) is not supported** by graw2root.C — a compact run yields 0 events with only a generic warning.
- **Pedestal window vs signal** — if the signal reaches cells 1–20, the pedestal is biased high and the strip fades; move the window with the trigger delay.

macros/{graw2root, analyzeUVW, makeTracks, analyzeTracks}.C · validated against TPCReco/simplefit: median axis diff